Abhay Kiran M

261100620002

Shared_pthread

```
/*
 * Program: Shared versus Private Data using POSIX Pthreads
 *
 * Description:
 * 1. Creates two threads using pthread_create().
 * 2. Both threads access the same global variable 'shared'.
 * 3. Each thread has its own local variable 'mine'.
 * 4. getpid() returns the same process ID for both threads.
 * 5. pthread_self() returns a unique thread ID for each thread.
 * 6. Prints the addresses of shared and private variables.
 * 7. pthread_join() waits for both threads to complete.
 */

#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

// Global variable shared by all threads
int shared = 100;

// Thread function
static void *worker(void *arg)
{
   (void)arg;

   // Private local variable for each thread
   int mine = 1;

   // Get process ID
   pid_t pid = getpid();

   // Get current thread ID
   pthread_t me = pthread_self();

   // Print process and thread IDs
   printf("pid=%d thread=%lu\n",
         pid, (unsigned long)me);
```

```
    // Print addresses of shared and private variables
    printf("  &shared=%p  &mine=%p\n",
           (void *)&shared, (void *)&mine);

    // Keep both threads alive briefly
    sleep(1);

    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    // Create first thread
    pthread_create(&t1, NULL, worker, NULL);

    // Create second thread
    pthread_create(&t2, NULL, worker, NULL);

    // Wait for first thread to finish
    pthread_join(t1, NULL);

    // Wait for second thread to finish
    pthread_join(t2, NULL);

    printf("Both threads completed.\n");

    return 0;
}
```

```
$ gcc assign.c
$ ./a.out
pid=551 thread=136479258961600
  &shared=0x63f680c00010  &mine=0x7c208dbfdea8
pid=551 thread=136479267354304
  &shared=0x63f680c00010  &mine=0x7c208e3feea8
Both threads completed.
```

Fork_print

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
```

```
    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
    } else {
        // Parent process
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);
        wait(NULL);
        printf("Parent: child has exited\n");
    }

    return 0;
}
```

```
$ gcc assign.c
$ ./a.out
Parent: my pid = 777, child pid = 778
Child: my pid = 778, parent pid = 777
Parent: child has exited
$
```

Fork_sleep

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
        printf("Child: sleeping for 20 seconds...\n");
        fflush(stdout);
        sleep(20);
        printf("Child: woke up, exiting now\n");
    } else {
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);
        printf("Parent: waiting for child to finish...\n");
        fflush(stdout);

        int status;
```

```
        pid_t exited_pid = waitpid(pid, &status, 0);

        if (exited_pid == pid) {
            if (WIFEXITED(status))
                printf("Parent: child %d exited with status %d\n", exited_pid,
    WEXITSTATUS(status));
            else
                printf("Parent: child %d did not exit normally\n", exited_pid);
        }
    }

    return 0;
}
```

```
$ gcc assign.c
$ ./a.out
Parent: my pid = 818, child pid = 819
Parent: waiting for child to finish...
Child: my pid = 819, parent pid = 818
Child: sleeping for 20 seconds...
Child: woke up, exiting now
Parent: child 819 exited with status 0
$
```

Fork_three_children

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int sleep_times[3] = {10, 20, 30};
    pid_t child_pids[3];

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
            return 1;
        } else if (pid == 0) {
            printf("Child %d: pid = %d, sleeping for %d seconds\n",
                i, getpid(), sleep_times[i]);
            fflush(stdout);
            sleep(sleep_times[i]);
            printf("Child %d: pid = %d, woke up after %d seconds\n",
                i, getpid(), sleep_times[i]);
            return 0;
```

```
        } else {
            child_pids[i] = pid;
        }
    }

    printf("Parent: pid = %d, waiting for all children...\n", getpid());
    fflush(stdout);

    for (int i = 0; i < 3; i++) {
        int status;
        pid_t exited_pid = waitpid(child_pids[i], &status, 0);
        if (exited_pid == child_pids[i] && WIFEXITED(status)) {
            printf("Parent: child %d (pid %d) exited with status %d\n",
                   i, exited_pid, WEXITSTATUS(status));
        }
    }

    printf("Parent: all children have exited\n");
    return 0;
}
```

```
$
$ gcc assign.c
$ ./a.out
Child 0: pid = 852, sleeping for 10 seconds
Child 1: pid = 853, sleeping for 20 seconds
Parent: pid = 851, waiting for all children...
Child 2: pid = 854, sleeping for 30 seconds
Child 0: pid = 852, woke up after 10 seconds
Parent: child 0 (pid 852) exited with status 0
Child 1: pid = 853, woke up after 20 seconds
Parent: child 1 (pid 853) exited with status 0
Child 2: pid = 854, woke up after 30 seconds
Parent: child 2 (pid 854) exited with status 0
Parent: all children have exited
$
```

Fork_waitpid

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
    } else {
```

```
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);

        int status;
        pid_t exited_pid = waitpid(pid, &status, 0);

        if (exited_pid == pid) {
            if (WIFEXITED(status))
                printf("Parent: child %d exited with status %d\n", exited_pid,
    WEXITSTATUS(status));
            else
                printf("Parent: child %d did not exit normally\n", exited_pid);
        }
    }

    return 0;
}
```

```
$ gcc assign.c
$ ./a.out
Parent: my pid = 915, child pid = 916
Child: my pid = 916, parent pid = 915
Parent: child 916 exited with status 0
$
```

Rec_fork

```
    #include <stdio.h>
    #include <unistd.h>
    #include <sys/wait.h>

    int main() {
        printf("Original parent: pid = %d\n", getpid());
        fflush(stdout);

        pid_t pid1 = fork();

        if (pid1 < 0) {
            perror("fork failed");
            return 1;
        } else if (pid1 == 0) {
            // ---- child1 ----
            printf("Child1: pid = %d, parent pid = %d\n", getpid(), getppid());
            fflush(stdout);

            pid_t pid2 = fork();

            if (pid2 < 0) {
                perror("fork failed");
```

```
            return 1;
        } else if (pid2 == 0) {
            // ---- child2 ----
            printf("Child2: pid = %d, parent pid = %d\n", getpid(), getppid());
            fflush(stdout);

            pid_t pid3 = fork();

            if (pid3 < 0) {
                perror("fork failed");
                return 1;
            } else if (pid3 == 0) {
                // ---- child3 ----
                printf("Child3: pid = %d, parent pid = %d, sleeping 10s\n", getpid(),
getppid());
                fflush(stdout);
                sleep(10);
                printf("Child3: pid = %d, woke up, exiting\n", getpid());
                return 0;
            } else {
                int status;
                waitpid(pid3, &status, 0);
                printf("Child2: pid = %d, child3 (pid %d) exited\n", getpid(), pid3);
                sleep(10);
                printf("Child2: pid = %d, woke up, exiting\n", getpid());
                return 0;
            }
        } else {
            int status;
            waitpid(pid2, &status, 0);
            printf("Child1: pid = %d, child2 (pid %d) exited\n", getpid(), pid2);
            sleep(10);
            printf("Child1: pid = %d, woke up, exiting\n", getpid());
            return 0;
        }
    } else {
        // ---- original parent ----
        int status;
        waitpid(pid1, &status, 0);
        printf("Original parent: pid = %d, child1 (pid %d) exited\n", getpid(), pid1);
    }

    return 0;
```

}

```
$ gcc assign.c
$ ./a.out
Original parent: pid = 946
Child1: pid = 947, parent pid = 946
Child2: pid = 948, parent pid = 947
Child3: pid = 949, parent pid = 948, sleeping 10s
Child3: pid = 949, woke up, exiting
Child2: pid = 948, child3 (pid 949) exited
Child2: pid = 948, woke up, exiting
Child1: pid = 947, child2 (pid 948) exited
Child1: pid = 947, woke up, exiting
Original parent: pid = 946, child1 (pid 947) exited
$
```

Fork_exec

```
#include <stdio.h>
#include <unistd.h>

int main() {
   printf("exec_prog: pid = %d, starting count\n", getpid());
   fflush(stdout);

   for (int i = 1; i <= 10; i++) {
      printf("%d\n", i);
      fflush(stdout);
      sleep(1);
   }

   return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
   printf("fork_exec: parent pid = %d\n", getpid());
   fflush(stdout);

   pid_t pid = fork();

   if (pid < 0) {
      perror("fork failed");
      return 1;
   } else if (pid == 0) {
      printf("fork_exec: child pid = %d, about to exec\n", getpid());
      fflush(stdout);
```

```
            execl("./exec_prog", "exec_prog", NULL);

            perror("execl failed");
            return 1;
        } else {
            int status;
            waitpid(pid, &status, 0);
            printf("fork_exec: child (pid %d) exited\n", pid);
        }

        return 0;
}
```

```
$ gcc -o exec_prog exec_prog.c
$ gcc rec.c
$ ./a.out
fork_exec: parent pid = 1140
fork_exec: child pid = 1141, about to exec
exec_prog: pid = 1141, starting count
1
2
3
4
5
6
7
8
9
10
fork_exec: child (pid 1141) exited
$
```